Technical Explanation Report

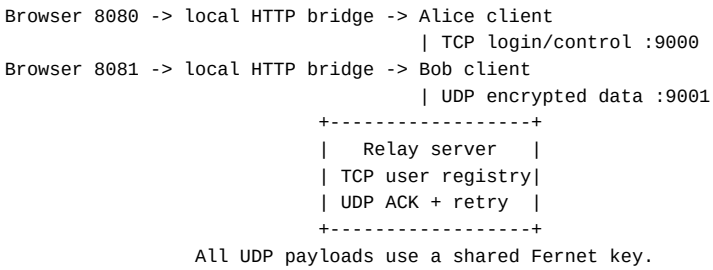
Assignment 1: Distributed and Streaming-based Client-Server Collaboration Application

Prototype: Collab LAN | Python standard-library sockets + cryptography | October 2026

1. Objective and scope

Build a self-hosted collaboration application for 10-20 concurrent users on a LAN: login and active-user discovery, unicast/broadcast messaging, reliable binary file transfer over UDP, resilience and encryption. The demo includes browser UI and CLI. The implementation is a reproducible academic prototype; its 20-client load target and WAN/loss resilience still require additional test evidence.

2. System architecture



TCP: a 4-byte big-endian frame length followed by UTF-8 JSON. UDP: encrypted JSON (Fernet). The shared key is supplied via COLLAB_KEY. An unguessable per-session token from the TCP welcome must be attached to each client UDP packet. This provides session authorization but is not credential-based authentication or individual key management.

3. Protocol specification

Message	Transport	Essential fields / meaning
login	TCP	type, name
welcome	TCP	type, token, udp_port
heartbeat	UDP	type, from, token
presence	UDP	type, names[]
data(chat)	UDP	id, from, to, kind, payload
data(file_start)	UDP	id, kind, meta:{transfer,filename,count,sha256}
data(file_chunk)	UDP	id, kind, meta:{transfer,index}, payload=base64
data(file_end)	UDP	id, kind, meta:{transfer}
ack	UDP	type, id, from, token (client); id only (server)

Maximum encrypted UDP frame: 1100 bytes; file chunk: 320 bytes raw (base64 in JSON) to avoid common IP fragmentation. Each packet has a unique ID. The sending client tracks up to 6 in-flight packets and resends only unacknowledged packets after 250 ms, up to 15 attempts. The server independently forwards packets to recipients and retries at 250 ms up to 12 attempts; receivers deduplicate packet IDs and ACK before processing. The server currently does not persist offline messages.

4. Client handshake sequence

```
Client                      Server
 | -------- TCP connect -----> |
 | -------- login(name) -----> |
 | <------ welcome(token) ---- |
 | ------ UDP heartbeat -----> |
 | <------ presence(names) --- |
 | ----- UDP data(chat) -----> |
 | <--------- ACK ------------ |
 |             server forwards -> Recipient
 |                                Recipient -> server ACK
```

5. File-transfer sequence

```
Sender                    Relay                    Receiver
  | file_start + checksum -> |  file_start ------->  |
  | <------ ACK ----------   |               ACK <- |
  | file_chunk[0..N-1] ---> |  chunks --------->    |
  | <---- ACK per chunk ---  |     receiver ACKs each|
  | retransmit missing ---> |                       |
  | file_end ------------->  |  file_end -------->   |
  |                          |   assemble ordered chunks
  |                          |   verify SHA-256; save file
```

For file data, payload bytes are base64-encoded into encrypted UDP JSON datagrams. A transfer ID and monotonically numbered chunk index preserve ordering after UDP reordering. The end marker is not proof of application-level completion: only a file_complete event at the receiver confirms verified assembly. Empty files are supported.

6. Implementation steps and code mapping

1) Protocol design: common.py defines framing, serialization and Fernet; data messages contain unique IDs and types.

2) Handshake: server.py / client.py use socket TCP connect, login and welcome.

3) State: server.py stores connected usernames, token and last UDP endpoint.

4) Concurrency: thread per TCP session, UDP loop and retransmit loop; independent client listener and heartbeat.

5) Messaging: route() forwards broadcast and private encrypted UDP datagrams.

6) File transfer: client.py splits binary content, sends start/chunks/end, verifies SHA-256.

7) Resilience: socket timeout handling, reconnect after logout, UDP ACK/retries, duplicate suppression.

8) Encryption: common.py wraps UDP JSON with Fernet (AES + HMAC).

9) UI: webui.py serves JavaScript interface with chat, presence, file progress and upload.

10) Testing: tests/test_integration.py runs isolated TCP/UDP end-to-end tests.

7. Challenges and solutions

Packet size: Fernet, JSON and base64 add overhead. During the initial 125 KB binary-file test, a 480-byte chunk exceeded the configured 1100-byte UDP frame ceiling. The chunk size was reduced to 320 bytes; all five integration tests subsequently passed.

Out-of-order arrival: a file is identified by transfer ID; chunk indices are stored in a dictionary, then reassembled in index order. A SHA-256 check rejects corruption.

Reliability and duplicates: unique packet IDs, sender window, retries, recipient ACK and duplicate suppression. Note that acknowledging server ingress differs from acknowledging end-recipient completion.

UI integration: the browser communicates with a localhost Python HTTP bridge, and only the bridge handles raw sockets. The UI does not rely on a WebSocket library or a Flask/Django networking framework.

8. Test results and reproducibility

Command: `python -m unittest discover -s tests -v`. Observed result in this environment: 5 tests, 5 passed. Verified tests: presence/private+broadcast chat; 125,000-byte pseudorandom file SHA-256 match; empty file; duplicate login rejection; disconnect cleanup. These tests are localhost functional tests, not measurements under 20 clients or injected packet loss.

9. Remaining work and deviations

The brief contains a transport inconsistency: its goal describes custom UDP streaming, its TCP handshake criterion requires TCP, and the techniques section discusses TCP reliable transfer. This design uses TCP control/handshake plus UDP message and file transport. Confirm with instructor. Password-based account authentication, full BBR/CUBIC congestion algorithms, streaming disk I/O, stronger sender-to-recipient delivery receipts, 20-client stress validation and loss-injection evidence are not implemented. Do not claim these as completed rubric items. The baseline HTTP UI is for local use only; this prototype is not hardened for public internet deployment.

10. Submission and demo instructions

Run server in one terminal and two clients with different usernames in separate terminals, following README.md. Demonstrate active list, private chat, global chat, binary file received and its SHA-256, logout, and automated tests. Present the architecture diagram, packet fields and known limitations. Python dependencies are in requirements.txt.